1. What does the database actually need to do?

Before choosing PostgreSQL, MongoDB or anything else, identify the data and access patterns.

My application has three core entities:

• User: Someone who creates or receives access to documents.

• Document: The document's title, formatted content, owner and timestamps.

• DocumentShare: A permission linking a user to someone else's document.

I will use one concrete example throughout.

Alice creates a document called Project Proposal. She shares it with Bob as an editor and Carol as a viewer.



The database must answer these questions efficiently:

User action	Database operation
Alice creates a document	INSERT a document
Alice opens it	SELECT document by ID
Alice edits content	UPDATE document content
Alice renames it	UPDATE document title
Alice shares with Bob	INSERT a share record

User action	Database operation
Bob views shared documents	SELECT documents shared with Bob
Carol attempts to edit	Check role and reject
Alice deletes the document	DELETE document and related shares

These operations are my access patterns. They should influence my database and index choices.

Functional vs nonfunctional database requirements

Functional	Nonfunctional
Store users and documents	Data must survive refreshes and deployments
Preserve rich-text formatting	Reads and writes should be reasonably fast
Support sharing and roles	Unauthorized access must be prevented
List owned and shared documents	Data integrity must be maintained
Support updates and deletion	Concurrent edits should have defined behavior

For the take-home, I don't have a specified user count, request rate or document-size distribution. So I should avoid pretending I have precise capacity requirements.

2. Which database should I choose?

My first decision is SQL vs NoSQL.

Option A: PostgreSQL — selected

PostgreSQL is a relational database. I can define tables, primary keys, foreign keys, unique constraints and transactions.

It also supports jsonb, which is useful because my rich-text editor produces structured JSON.

Option B: MongoDB

MongoDB stores flexible BSON documents. I could embed document content and sharing information together, or use separate collections.

It is a valid option, particularly if document structures evolve frequently.

Option C: SQLite

SQLite is an embedded relational database stored in a file. It is excellent for local development and small applications.

However, a local SQLite file is not durable on Vercel's ephemeral serverless filesystem. A managed remote SQLite service would be a different deployment option.

Option D: Firebase Firestore

Firestore is a managed document database with real-time listeners and a security-rules system.

It could simplify some real-time features, but I would adopt its query model, access-control rules and vendor-specific SDKs.

Comparison

Criteria	PostgreSQL	MongoDB	SQLite	Firestore
Relational joins	Native	$lookup / modeling alternatives	Native	No SQL-style joins
Foreign keys	Yes	Not database-enforced	Yes, when enabled	No
JSON content	JSONB	Native BSON	JSON functions	Map/document fields
Transactions	Yes	Yes	Yes	Yes, with limits
Serverless deployment	Hosted service	Hosted service	Needs durable remote storage	Managed
Fit for users/documents/shares	Excellent	Good	Good	Good

Why PostgreSQL wins for my use case

The deciding factor is not that SQL is universally better than NoSQL.

My data has important relationships and integrity rules:

1. Every document has one owner.

2. A share refers to an existing document.

3. A share refers to an existing user.

4. The same user should not have duplicate share records for the same document.

5. Deleting a document should clean up its shares.

PostgreSQL can enforce these through foreign keys and unique constraints.

Tradeoff: PostgreSQL introduces a separate database service and requires schema migrations. I accept that because it provides strong data integrity and supports my queries naturally.

Interview answer: "I chose PostgreSQL because the application's core data is relational, and I need transactional updates and referential integrity. I can still store flexible rich-text content in a JSONB column, so I get structured relationships without forcing the editor document into rigid relational tables."

3. How should I model the data?

I will start with a deliberately simple design.

First attempt: Put everything in one table

```
CREATE TABLE documents (
    id UUID PRIMARY KEY,
    title TEXT,
    content JSONB,
    owner_email TEXT,
    shared_with_emails TEXT[]
);
```

This looks easy. But consider the problems.

• A user's email can change.

• I can't easily represent viewer vs editor permissions.

• Duplicate email addresses may appear in the array.

• I cannot use a normal foreign key to validate every email inside the array.

• Querying documents shared with a user becomes less straightforward.

For a throwaway prototype, it might work. But my application explicitly evaluates sharing behavior.

Better approach: Three tables



A user owns many documents. Shares link users and documents.

Foreign keys enforce valid references; unique share pairs prevent duplicates.

content — JSONB

created_at

updated_at

document_shares

id — PK

document_id — FK

user_id — FK

role

created_at

DocumentShare is a junction table implementing a many-to-many relationship between users and documents.

Why is owner_id stored directly in the document table instead of creating a share row for the owner?

Because ownership is different from sharing. Every document must have exactly one owner, whereas it can have zero or many shared users.

A non-null owner_id makes ownership explicit and makes the "My documents" query simple.

4. Actual PostgreSQL schema

This is an illustrative SQL schema for the proposed architecture, not a migration we've executed.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TYPE share_role AS ENUM ('VIEWER', 'EDITOR');

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id UUID NOT NULL REFERENCES users(id),
    title VARCHAR(200) NOT NULL,
    content JSONB NOT NULL DEFAULT
        '{"type":"doc","content":[{"type":"paragraph"}]}'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT document_title_not_blank
        CHECK (length(trim(title)) > 0)
);

CREATE TABLE document_shares (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID NOT NULL
        REFERENCES documents(id) ON DELETE CASCADE,
    user_id UUID NOT NULL
        REFERENCES users(id) ON DELETE CASCADE,
    role share_role NOT NULL DEFAULT 'VIEWER',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT unique_document_share
        UNIQUE (document_id, user_id)
);
```

Understand every important decision

Schema feature	Why it exists
UUID primary keys	Globally unique identifiers, convenient for distributed ID generation
NOT NULL	Prevents missing required values
UNIQUE(email)	Prevents duplicate user emails
Foreign keys	Prevent dangling references
JSONB	Stores structured TipTap content

Schema feature	Why it exists
ON DELETE CASCADE	Automatically removes share records when a document is deleted
UNIQUE(document_id, user_id)	Prevents duplicate shares
Role enum	Restricts permissions to supported values
TIMESTAMPTZ	Stores timestamps with timezone-aware semantics

There are two important subtleties.

First, updated_at DEFAULT NOW() only sets the initial value. PostgreSQL does not automatically update this column on every modification. My application must explicitly set it, or I can add a database trigger.

Second, the database cannot automatically prove that a JSONB value is a valid TipTap document merely because its column type is JSONB. I need application-level schema validation.

The database also doesn't currently prevent sharing a document with its own owner. That rule requires an application check or an additional database mechanism.

5. Why JSONB instead of HTML or Markdown?

Suppose Alice types:

Project Proposal

This is an important document.

TipTap might represent the content like this:

```
{
  "type": "doc",
  "content": [
    {
      "type": "heading",
      "attrs": { "level": 1 },
      "content": [
        { "type": "text", "text": "Project Proposal" }
      ]
    },
    {
      "type": "paragraph",
      "content": [
        { "type": "text", "text": "This is an " },
        {
          "type": "text",
          "text": "important",
          "marks": [{ "type": "italic" }]
        },
        { "type": "text", "text": " document." }
      ]
    }
  ]
}
```

The JSON contains both the text and the formatting structure.

Storage alternatives

Format	Advantages	Disadvantages
TipTap JSON in JSONB	Preserves editor structure; direct load/save	Editor-schema coupling; text search requires extra work
HTML	Easy to render/export	Requires careful sanitization and parsing
Markdown	Portable, readable	Some formatting cannot round-trip exactly
Plain text	Simplest	Loses formatting
CRDT document	Supports merging concurrent edits	More complex persistence and collaboration logic

I choose TipTap JSON because formatting preservation is a core requirement and TipTap is my editor.

However, storing JSON does not automatically eliminate XSS risks. Imported content must still be validated, and rendering must avoid unsafe HTML injection.

6. Actual SQL queries for the main user flows

This is the most useful part for interviews: translating product behavior into database operations.

A. Alice creates a document

```
INSERT INTO documents (owner_id, title)
VALUES ($1, 'Untitled Document')
RETURNING id, title, content, created_at;
```

$1 is the authenticated user's ID, supplied by the server, not trusted from a browser request body.

Why? If the client could submit arbitrary owner_id values, Bob could create documents pretending to be Alice.

B. Alice lists her documents

```
SELECT id, title, created_at, updated_at
FROM documents
WHERE owner_id = $1
ORDER BY updated_at DESC
LIMIT 50;
```

Notice that I do not select content.

The document list only needs titles and timestamps. Fetching every document's full JSON content would waste database bandwidth and application memory.

This is a simple example of projection: retrieving only the columns needed.

C. Bob lists documents shared with him

```
SELECT
    d.id,
    d.title,
    d.updated_at,
    ds.role,
    u.name AS owner_name
FROM document_shares ds
JOIN documents d ON d.id = ds.document_id
JOIN users u ON u.id = d.owner_id
WHERE ds.user_id = $1
ORDER BY d.updated_at DESC
LIMIT 50;
```

Here, JOIN connects the share record to the document and its owner.

This is why relational modeling fits my use case naturally.

D. Bob opens a document

A naive implementation would query the document first and check permissions later.

I can instead combine retrieval with the access condition:

```
SELECT d.*
FROM documents d
WHERE d.id = $1
  AND (
    d.owner_id = $2
    OR EXISTS (
      SELECT 1
      FROM document_shares ds
      WHERE ds.document_id = d.id
        AND ds.user_id = $2
    )
  );
```

$1 is the document ID and $2 is the current user's ID.

If the user is neither the owner nor an invited user, the query returns no row.

The API can return 404 Not Found so unauthorized users cannot distinguish a nonexistent document from a private one.

E. Alice saves document content

The simplest approach is:

```
UPDATE documents
SET
    content = $1::jsonb,
    updated_at = NOW()
WHERE id = $2
RETURNING id, updated_at;
```

But this query is not sufficient for a real application.

It does not check whether the caller can edit the document.

I should include authorization in the update condition:

```
UPDATE documents d
SET
    content = $1::jsonb,
    updated_at = NOW()
WHERE d.id = $2
  AND (
    d.owner_id = $3
    OR EXISTS (
      SELECT 1
      FROM document_shares ds
      WHERE ds.document_id = d.id
        AND ds.user_id = $3
        AND ds.role = 'EDITOR'
    )
  )
RETURNING d.id, d.updated_at;
```

Now the database only updates the document if the current user is its owner or an editor.

This is a powerful pattern: authorize the mutation at the point of mutation, rather than relying solely on a separate check performed earlier.

7. Why permission checks must be server-side

Imagine Bob is a viewer. The React UI hides the Edit button.

Could Bob still send an HTTP PATCH request manually?

Yes.

The browser is not a trusted security boundary. Anyone can use developer tools, scripts or an API client to send requests.

Therefore:

• The frontend controls user experience.

• The backend enforces business permissions.

• The database enforces referential integrity and constraints.

I should centralize authorization logic in lib/access.ts, but avoid a race between checking permission and performing a mutation.

For example:

1. Bob is currently an editor.

2. The backend checks his permission.

3. Alice revokes Bob's access.

4. Bob's previously authorized update executes.

If the check and update are separate operations, a race is possible.

An authorization condition inside the update statement, or an appropriate transaction with locking, helps close that gap.

Sharing query

When Alice shares with Bob:

```
INSERT INTO document_shares (
    document_id,
    user_id,
    role
)
SELECT d.id, $2, 'EDITOR'
FROM documents d
WHERE d.id = $1
  AND d.owner_id = $3
  AND d.owner_id <> $2
ON CONFLICT (document_id, user_id)
DO UPDATE SET role = EXCLUDED.role
RETURNING *;
```

This allows only the owner to grant access and avoids sharing with the owner.

The foreign key ensures the recipient user exists. The unique constraint prevents duplicate share rows.

8. Database indexes: why and when?

An index is a data structure that helps PostgreSQL locate matching rows without scanning the entire table.

Think about a book.

Without an index, finding a topic might require reading every page. With an index, I jump directly to relevant pages.

However, indexes are not free. They consume storage and must be maintained during writes.

My expected access patterns

Query	Useful index
Find document by ID	Document primary key
Find user by email	Unique email index
List documents owned by user	(owner_id, updated_at DESC)
Find share for document and user	Unique (document_id, user_id)
List documents shared with user	(user_id, document_id)

Suggested additional indexes:

```
CREATE INDEX idx_documents_owner_updated
ON documents (owner_id, updated_at DESC);

CREATE INDEX idx_shares_user_document
ON document_shares (user_id, document_id);
```

The unique constraint on (document_id, user_id) already creates an index in PostgreSQL, so I don't need to duplicate it.

Why not index everything?

Because every INSERT, UPDATE or DELETE may also need to update relevant indexes.

For example, indexing updated_at improves some sorting/filtering queries but creates index maintenance work on every autosave.

For this small application, the overhead is unlikely to be significant. At scale, I would measure with EXPLAIN ANALYZE and actual production query patterns.

9. Transactions and consistency

Suppose Alice deletes a document.

I need to remove:

• The document itself.

• All its sharing records.

My schema uses:

```
REFERENCES documents(id) ON DELETE CASCADE
```

So PostgreSQL handles share cleanup as part of the document deletion transaction.

```
DELETE FROM documents
WHERE id = $1
  AND owner_id = $2
RETURNING id;
```

This deletes the document only if the current user owns it.

The related shares are deleted automatically.

What is a transaction?

A transaction groups operations into one atomic unit.

Either all required operations succeed, or none of them are committed.

For example, imagine I later introduce a document_versions table. Saving a document might need to update the current document and insert a history record.

I would want both changes to succeed together.

```
BEGIN;

UPDATE documents
SET content = $1::jsonb,
    updated_at = NOW()
WHERE id = $2;

INSERT INTO document_versions (
    document_id,
    content
)
VALUES ($2, $1::jsonb);

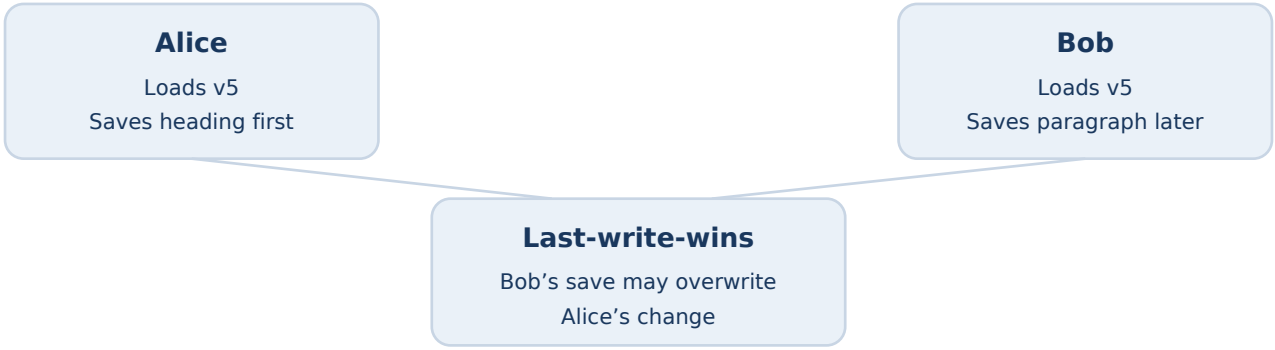
COMMIT;
```

This is only illustrative; document_versions is not part of my current schema. Production code would also check that the update affected exactly one authorized document before inserting the version.

10. Concurrency: what if Alice and Bob edit together?

This is one of the most important system design questions.

Suppose both Alice and Bob open document version 5.



Bob's full-document save may overwrite Alice's heading change, even though they edited different parts.

This is called a lost update.

Three possible solutions

Approach	Complexity	Behavior
Last write wins	Low	Later save overwrites earlier save
Optimistic concurrency control	Moderate	Rejects stale saves
CRDT/OT collaboration	High	Merges concurrent editing operations

My initial take-home design selected last-write-wins because simultaneous editing is not required.

But if I want a relatively small improvement, I can introduce a version number.

Optimistic concurrency control

Add:

```
ALTER TABLE documents
ADD COLUMN version INTEGER NOT NULL DEFAULT 1;
```

Then save only if the expected version matches.

```
UPDATE documents d
SET
    content = $1::jsonb,
    version = version + 1,
    updated_at = NOW()
WHERE d.id = $2
  AND d.version = $3
  AND (
    d.owner_id = $4
    OR EXISTS (
      SELECT 1
      FROM document_shares ds
      WHERE ds.document_id = d.id
        AND ds.user_id = $4
        AND ds.role = 'EDITOR'
    )
  )
RETURNING d.version;
```

If the update returns zero rows, the API needs to distinguish a version conflict from missing permissions or a nonexistent document.

A stale authorized save should receive 409 Conflict.

The frontend can then offer to reload, compare changes or preserve the unsaved draft.

Tradeoff: Optimistic locking prevents silent overwrites but does not automatically merge edits.

For actual simultaneous editing, we'd consider Yjs, WebSockets and a collaboration service.

11. How would the database scale?

I should not add Redis, read replicas or partitioning simply because they sound like production architecture.

First, identify the bottleneck.

Potential bottleneck	Possible response
Slow queries	Inspect execution plans and add targeted indexes
Too many database connections	Use connection pooling
High read traffic	Cache appropriate metadata or use read replicas
Large document content	Limit payload sizes; separate metadata and content if necessary
Frequent autosaves	Debounce, batch or change persistence strategy
Huge document table	Consider partitioning based on measured access patterns
Real-time collaboration	Add a dedicated synchronization service

Important: serverless database connections

Vercel may run many application instances in parallel.

If each instance opens too many direct PostgreSQL connections, the database can exhaust its connection limit.

A pooled connection endpoint or suitable serverless database driver helps manage this.

This is an important deployment consideration for Prisma and Neon.

Should I cache documents in Redis?

Not initially.

The application is small, and PostgreSQL can serve my current access patterns.

Caching editable document content introduces invalidation problems. If Alice saves a change, how quickly should Bob's cached copy become stale?

For a future production system, I might cache document metadata or frequently read public content. But caching is a response to measured load, not a default requirement.

12. Failure scenarios and recovery

A strong system design explanation includes what happens when something goes wrong.

Failure	Expected behavior
Database unavailable	API returns an error; UI preserves unsaved draft where possible
Save request times out	Show uncertain/failed save state and allow retry
Duplicate share request	Unique constraint prevents duplicate rows
User loses access during editing	Subsequent saves fail authorization
Two users save stale content	Last-write-wins initially; optimistic locking in improved design
Invalid TipTap JSON	Server rejects it before persistence
Document deleted while open	Subsequent reads/saves return not found
Network disconnects during editing	Keep local editor state and retry carefully

One subtle problem is retrying writes.

If the client sends a save, the database commits it, but the response is lost, the client may not know whether the save succeeded.

For my current full-document update, repeating the same content is generally safe, although timestamps may change. With version checks, retries need additional handling because a successful first attempt changes the version.

That is why robust autosave needs more than a debounce timer: it needs request ordering, status tracking and a retry strategy.